

OuicheFS – Project Report

Team 10 (Abhishek Jayachandran, Hong Yu Wang)

1. Design Decisions and Implementation

Task 1.2 – Bypassing VFS Page Cache

Instead of the **generic_file_read_iter** and **generic_file_write_iter**, we implemented custom read and write callbacks, and pointed the **file_operations** struct to them.

When a user program calls **ouichefs_read(fd, buffer, count)**, we use the offset to calculate the logical block the user wants by dividing the offset by the block size (4096). Then we use **sb_bread(sb, physical_block)** to fetch exactly one 4 KiB block from the physical disk. It returns a struct **bh_data** (a chunk of raw memory holding the disk data). We then use **copy_to_user()** to safely copy the requested bytes from the **bh_data->b_data** array directly into the user application's RAM. For the cleanup, we call **brelease(bh)** to release our lock on that memory chunk, and manually increment ***ppos** by the no of bytes we read.

When a user program calls **write(fd, buffer, count)**, we first check if the file was opened with the **O_APPEND** flag (like when someone types **echo "hello" >> file.txt**). If yes, jump ***ppos** to EOF (**inode->i_size**). If over-writing an existing block, we read it off the disk first using **sb_bread()**, so we don't accidentally erase the data surrounding it. If we are writing on a new block, we use **get_free_block()**. We use **copy_from_user()** to pull the data from the user's RAM and paste it into **bh_data->b_data**. We then call **mark_buffer_dirty(bh)** to inform the kernel of the edit. Finally, if we wrote past the old EOF, we update inode size, advance ***ppos** and return no. of bytes.

Task 1.3 – Extent array setup and IOCTL

We created a new 8-byte **ouichefs_extent** structure and replaced the old array in **ouichefs_file_index_block** with an array of these extents. We also implemented the **OUCHEFS_IOC_GET_EXTENTS** ioctl command and added a **ouichefs_ioctl** callback to the **file_operations** struct. When invoked, it takes the file descriptor, extracts the **file_inode()**, securely reads the inode's index block using **sb_bread()**, and iterates through the extent array. We make sure to use **le32_to_cpu()** to convert the raw Little-Endian data into CPU-native integers, check if **count==0** (breaks the loop if yes, because they are unallocated) before formatting and passing them to **printk**.

Task 1.4 – Extent Aware Logical Translation

For the read path, our **ouichefs_extent_get_block()** algorithm performs a linear scan through the 512-slot extent array, maintaining a cumulative sum of the **count** variables. If the **count == 0**, we hit an empty slot so return 0. If the logical block the user requests is less than the **cumul** plus this extent's **count**, then the logical block is inside this extent. Then we can calculate the offset difference with the formula **Physical Block = extent.start + (logical_block - cumulative_count)**. Finally, we went back to the **ouichefs_read** function and replaced the old array lookup with a call to the **ouichefs_extent_get_block()** helper, which will be invoked when calling **sb_bread()**.

Task 1.5 – Greedy Appending and Extent Eviction

For the write path, we implemented a "greedy append" algorithm: If the hard drive gives us a block that is adjacent to our last block, rather than wasting a new array slot, we just extend the current extent to cover it. Also because an extent now represents a range of blocks, the old file deletion logic which only freed one block per slot was causing permanent memory leaks.

When the write function allocates a new physical block, we loop through the extent array to find the very last active extent. We then check for contiguity with **last.start + last.count == new_block** check. If yes, instead of creating a new extent, we just increment the count of the existing one. If the check fails, then that means the file is fragmented, and we are forced to consume the next empty slot in the array and initialize it with **[start=phys_block, count=1]**.

When a user types `rm file.txt`, the VFS calls `ouichefs_evict_inode` to destroy the file and return its blocks to the global free bitmap. Because an extent now holds multiple blocks, we use a nested loop to clean them up properly. The outer loop iterates through all 512 slots of the `index_block`. For each active slot, we extract the start and count. The inner loop then runs `count` times. Inside this, we call the kernel's `put_block()` function to free `start + 0`, `start + 1`, `start + 2`, etc. `inode->i_blocks += (sb->s_blocksize >> 9)` tells the VFS layer to shrink the file's size by 8 (because one block is 4KiB and VFS communicates in units of 512 bytes, so $4\text{KiB}/512 = 8$)

Tasks 1.6 – Contiguous Block Allocator

The `get_free_block()` function in our write was not smart. It only looked for one single block at a time. If we tried to write a 5 MB file (requiring 1,280 blocks), `get_free_block()` would run 1,280 separate times. If the disk was even slightly fragmented, it would provide scattered blocks and the Greedy Append check (`last.start + last.count == new_block`) would fail.

We therefore deleted it and built a new function `ouichefs_alloc_contiguous`. We scan the `bfree_bitmap` from block 1 (0 is superblock) and note the start and number of contiguous free blocks (`cur_start` and `cur_len`). For a First-Fit, Best-Effort approach, we:

- 1) break the loop if we find the exact matching number of contiguous free blocks as the `requested` size
- 2.) stop the current streak when it hits a "used" block. Before resetting `cur_len` to 0, check if this streak was the biggest seen so far. If yes, it saves it in a `best_len` variable. This ensures that even if we can't get all 1,280 blocks, the allocator will still give the largest physical chunk it could find on the disk.

Once the loop finishes, we claim the blocks so no one else takes them. We run a quick loop over the chosen bits in `bfree_bitmap` and flip them to "used" (using `clear_bit`). For the free block bitmap, because bitmap was loaded with `kzalloc()` and since `ouichefs_sync_fs()` periodically flushes RAM changes to the hard drive, we do not need to mark the buffer dirty. We update the global counter: `sbi->nr_free_blocks -= best_len`, write the starting block number into the `*block` pointer, and return the `best_len`.

Finally we plug this function back into the `ouichefs_write()` function. Now, after calculating exactly how many blocks the user's payload requires (with `requested = (count + block_size - 1) / block_size`), we can pass that number to `ouichefs_alloc_contiguous()` to get the location of the contiguous blocks.

Tasks 1.7 – Write-Time Reservations and Global GC

In order to prevent fragmentation and interleaved files by concurrent writers on the disk, we introduce write-time reservations. We updated the VFS in-memory inode structure in `ouichefs.h` to include two new fields:

`i_reserved_start` and `i_reserved_count`. We put them in-memory because reservations are temporary. If the system crashes, we don't want the disk to permanently think those blocks are used.

When `ouichefs_write()` needs a new block, first, we call `spin_lock()` so that if two threads try to write to the same file simultaneously, the spinlock freezes one thread so they don't corrupt the subsequent calculations. We then check if `i_reserved_count > 0`. If yes, take one block (`i_reserved_start`), increment the start, decrement the count, unlock the spinlock, and write the data. No disk scanning is required. If the stash is empty, we call `ouichefs_alloc_contiguous()`. Instead of asking for what we need, we ask for the requested amount plus the `reservation_size` (default 8). We use the blocks we need right now, and put the leftovers into `i_reserved_start` and `i_reserved_count` before unlocking the spinlock.

If the hard drive is almost full and many open files are holding onto their reserved blocks, the disk will throw an Out of Space (ENOSPC) error. To fix this, we use a garbage collector. `ouichefs_run_gc()` is invoked when the allocator returns 0. It takes the `sb->s_inode_list_lock` to safely freeze the kernel's global list of open files. It loops through every single open inode on the system. For each inode, it grabs that specific file's `i_lock`, zeroes out `i_reserved_start` and `i_reserved_count`, and returns those blocks to the global bitmap. Once the loop finishes, it unlocks the global list and retries the allocation.

Finally, consider the case if a user deletes a file, but that file still has some unused blocks sitting in its RAM stash. If they are not cleaned, they are leaked forever. To tackle this, in `super.c` inside `ouichefs_evict_inode()`, we grab the `i_lock`, extract the unused `r_count` and `r_start`, zero out the stash, unlock the spinlock, and then run a for loop calling `put_block()` to return them to the disk.

Task 1.8 – Sysfs statistics

To know the total number of extents and reserved blocks across the entire filesystem, we have to look at every single file. But if we lock every single file in the VFS cache to read its stats, we completely freeze the filesystem for all users. To prevent this we split the data gathering into two phases: the volatile data and the persistent data.

For the volatile data (memory scan): To find the blocks sitting in RAM stashes (**reserved_blocks**), we lock **sb->s_inode_list_lock** to freeze the global list of open files so the kernel doesn't delete a file while you are looking at it. Then loop through **sb->s_inodes**. For each file, we grab its individual **i_lock**, add its **i_reserved_count** to the total, and unlock it.

For the persistent data (disk scan): To find out the **max_file_size**, **total_extents**, and fragmentation, we bypassed the VFS Cache. Instead of looking at RAM, we use a for loop to read the raw, physical 4 KiB inode blocks directly off the disk using **sb_bread()**. Because we are reading the raw disk data, we don't need any VFS locks! Active read/write workloads can continue running seamlessly in the background while we tally up the fragmentation math. We make sure to specifically ignore extents where **start == 0** so that sparse holes don't ruin our fragmentation statistics.

We use the macro **OUICHEFS_RO_ATTR** to generate the 9 **_show** functions.

We exposed **reservation_size** as a Read/Write attribute (0644 permissions). We wrote a custom **_store** function that intercepts when a user types **echo 16 > /sys/ouichefs/loop0/reservation_size**. It uses the kernel's string-to-integer function **kstrtouint()** to convert the text "16" into an integer and inject it into the allocation engine.

Finally, we used **kobject** to create the actual folders. In **ouichefs_sysfs_init** (called during **insmod**), we created the root **/sys/ouichefs/** folder. In **ouichefs_sysfs_sb_init** (called during **mount**), we create the specific partition folder (like **loop0**), and attach the array of attributes to it.

Task 1.9 – Sparse files and Array Shifting

For sparse files, we represent a hole as an extent with **start = 0** and a count larger than zero. We updated **ouichefs_extents_get_block()** so that if it sees a hole, it returns 0 (or **OUICHEFS_HOLE_BLOCK**). What happens if an application tries to read from a hole? In **ouichefs_read**, we added an interceptor. If **ouichefs_extents_get_block** returns 0, we do not do the **sb_bread()** hardware call. Instead, we use the kernel's **clear_user(buffer, count)** function. This injects fake zeroes directly into the user application's RAM.

Array Shifting: What if the user writes physical data directly into the center of a 500-block hole?

We take one extent slot and split it into three pieces:

[Remaining Hole Before][New Physical Data][Remaining Hole After]

To do this inside **ouichefs_write**, we used a complex array-shifting algorithm:

1. Capacity Check: First, we verify that adding 2 extra slots won't push the array past **OUICHEFS_MAX_EXTENTS** (512).
2. The Shift: We slide all subsequent extents to the right to make room for the new entries. We used standard C **memmove()** directly on the **bh->b_data** array (**index** is just a convenient wrapper on top of **bh->b_data**). We use **memmove** here, not **memcpy**. Because we are shifting an array inside itself, the memory regions overlap. **memcpy** would corrupt the array, but **memmove** uses a temporary buffer to safely slide the data.
3. Once the space is cleared, we insert the new physical block extent into the middle, and update the count variables for the two **start = 0** hole fragments surrounding it.
4. Call **mark_buffer_dirty(bh)** to alert the kernel to save the new layout to the disk.

Task 1.10 – Defragmentation Engine

The **ouichefs_defrag_file** handle defragmentation process, having two ways to trigger it: manually (via a new **IOCTL**) and automatically (via a sysfs **threshold**).

Assessment:

1. Loops through the file's current extent array and calculates the total logical size of the file.
2. If the file is already perfectly contiguous (only uses 1 extent slot), exits early to save CPU cycles.

3. If it is fragmented, it calls the **ouichefs_alloc_contiguous()** function asking for a single, massive physical chunk large enough to hold the entire file.

Data Migration:

1. Set up a loop to go block-by-block through the old extents to move the data to the new chunk.
2. For Real Data: We use **sb_bread()** to fetch the old block from the disk, and **sb_getblk()** to prepare the new block in RAM. We use **memcpy()** to copy the data between the two **bh->b_data** arrays, and then call **mark_buffer_dirty(bh_new)**.
3. For Holes (start == 0): If we just leave the new physical block alone, it might contain "dirty" data from some other deleted file, causing a security leak! So we intercept the hole, use **sb_getblk()** to grab the new block, and use **memset(bh_new->b_data, 0, sb->s_block_size)** to fill it with zeroes before marking it dirty.

Cleanup:

Run one last loop over the old extent array. You use **put_block()** to release all the scattered, fragmented physical blocks back to the global disk bitmap so other files can use them.

Manual and Auto Triggers:

1. Manual: We add **OUCHEFS_IOC_DEFRAG_FILE** to our IOCTL switch. This lets the **test_defrag.c** tool force a file to defragment on command.
2. Automatic: We attempted to make a background scanner (**ouichefs_check_and_run_defrag**) that runs at the end of the **write()** path. It checks the live Sysfs **defrag_threshold** we defined earlier. If a file's extent count exceeds that threshold, it intercepts the write and tries to automatically defragment the file in the background.

3. Implemented and Functional Features

- **Extent-Based Storage:** Successfully replaced the flat block limit with an Endian-safe 512-slot array.
- **Contiguous Block Allocation:** Functional First-Fit bitmap scanning supporting "Best-Effort" fallbacks.
- **Write-Time Reservations:** **spin_lock**-protected RAM caching to prevent interleaved fragmentation.
- **Global Garbage Collection:** Safe revocation of unused reservations across all VFS inodes upon **ENOSPC**.
- **Sparse File & Hole Punching:** Zero-fill reading and memory-safe extent array splitting (**memmove**).
- **Sysfs Global Statistics:** Read-only metrics (bypassing VFS locks via **sb_bread**) and writable tunables.
- **Online Manual Defragmentation:** IOCTL-driven block migration, consolidation, and dirty-block sanitization.

4. Implemented but Not Fully Functional / Known Issues

Auto-Defragmentation Scanner Synchronization

- Problem: The auto-defragmentation logic occasionally fails to trigger accurately when evaluating fragmentation immediately following a **write()** operation.
- Why we think it occurs: The **ouichefs_check_and_run_defrag** scanner evaluates fragmentation by reading raw inodes directly from the physical disk (**sb_bread**). However, it is invoked synchronously at the end of the custom **write** path. Because Linux buffers I/O, the dirty extent metadata created in RAM has not yet been flushed to the physical disk. Thus, the scanner reads stale on-disk metadata, calculates a healthy fragmentation percentage, and aborts.
- Potential Fixes: Delegating the defragmentation scan to an asynchronous kernel worker thread (**kworker**) that waits for the page cache flush, or invoking the scanner during the **release** file operation (when the file is closed). For our testing script, we tried forcing a bash **sync** command before evaluation.

5. Not Implemented

Task 1.11 – Bonus: Advanced Block Allocator

Task 1.11 was not implemented due to time constraints. Replacing the linear bitmap-based contiguous allocator with a complex mechanism like buddy-allocator could not be attempted. We decided to prioritize the challenges in the existing tasks 1.2 through 1.10 and think of mitigating edge-case scenarios that may arise within these tasks.

6. Tests and Instructions

The implementation was validated using a custom automated bash script (**extent.sh**) combined with two C-based IOCTL utilities (**test_ioctl.c**, **test_defrag.c**).

Instructions to Run: Compile the module via make. From the ouichefs directory, execute the testing suite with root privileges: **chmod +x extent.sh && sudo ./extent.sh**

Testing Sequence:

1. Contiguous Allocation: Writes 5 MB of data to an empty filesystem. Uses **test_ioctl** to verify the allocator successfully grouped the blocks into a single extent.
2. Sparse Holes: Writes 1 MB, seeks past 10 MB of empty space, and writes 1 MB. Verifies logical file size and extent layout (**start = 0**).
3. Hole Splitting (Forced Fragmentation): Writes a new 1 MB block directly into the center of the previously created hole. Uses **test_ioctl** to verify **memmove** safely split the hole and shifted the array into three logical pieces.
4. Sysfs Statistics: Executes a system **sync** to flush buffers, then reads variables from **/sys/ouichefs/** to verify mathematical accuracy against the global bitmap.
5. Manual Defragmentation: Triggers **test_defrag** on the highly fragmented sparse file. Verifies via IOCTL that the extent layout collapses back into a single contiguous physical run.
6. Auto-Defragmentation Tuning: Lowers **defrag_threshold** via sysfs, creates a fragmented file, issues a **sync**, and verifies the background scanner successfully intercepts and defragments the file.

7. Conclusion

This project successfully evolved OuicheFS from a naive block-by-block filesystem into an optimized, extent-based architecture capable of handling files up to ~1 GiB. But more importantly we evolved with the project, understanding various Linux kernel concepts as we solved challenges and brainstormed about different scenarios. This was a wonderful exercise, but we also felt that this project requires quite a lot of effort and even more time (than was given unfortunately :() in order to fully implement all required functionalities. However, overall we had fun and learned a lot from this project! Cheers to the devs including our professor who made OuicheFS! Thank you for this!